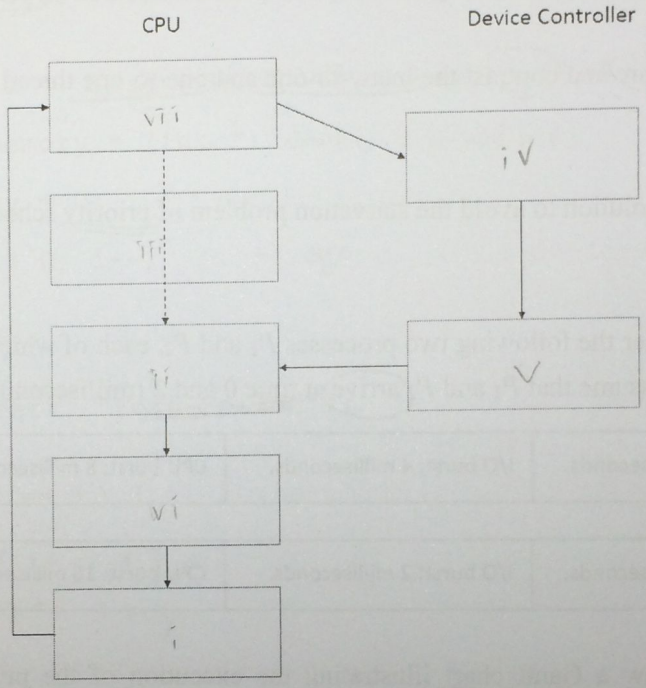
Operating Systems – Midterm
April 21, 2015

Total: 100 points

1. Interrupt and I/O operation
   (a) (5 points) What role do device controllers and device drivers play in a computer system?
   (b) (5 points) The interrupt-driven I/O cycle consists of (i) Input ready, output complete, or error generates interrupt signal, (ii) Interrupt handler processes data, return from interrupt, (iii) CPU executing checks for interrupts between instructions, (iv) device driver initiates I/O, (v) Controller initiates I/O, (vi) CPU resumes processing of interrupted process (task), and (vii) CPU receiving interrupt, transfer control to interrupt handler.. Organize the steps with the following flowchart.



2. (5 points) What are the advantages of using a higher-level language to implement an operating system?

3. (10 points) There are two different ways that commands can be processed by a command interpreter. One way is to allow the command interpreter to contain the code needed to execute the command. The other way is to implement the commands through system programs. Compare and contrast the two approaches.

4. System calls:
   (a) (10 points) Describe how a system call is invoked using an API function? You **need to** mention ***software interrupt***, ***dual-mode***, ***system call number (index)***, and ***table of code pointer***.
   (b) (5 points) Explain why programmers prefer programming according to API functions rather than invoking system calls directly.

create terminate

ready running

wait

5. Show the state of a process:
   (a) (2 points) after it is created (forked) by a (parent) process
   (b) (2 points) before it is selected by a short-term scheduler
   (c) (2 points) after it is selected by a short-term scheduler
   (d) (2 points) after it invokes a blocking system call
   (e) (2 points) after the blocking system call is complete
   (f) (2 points) after it is interrupted by a hardware interrupt
   (g) (3 points) after its time slice expired

6. Multithreaded Programming:
   (a) (5 points) Explain why allocating memory for process creation is costly as compared with thread creation.
   (b) (5 points) Compare and contrast the many-to-one and one-to-one thread models.

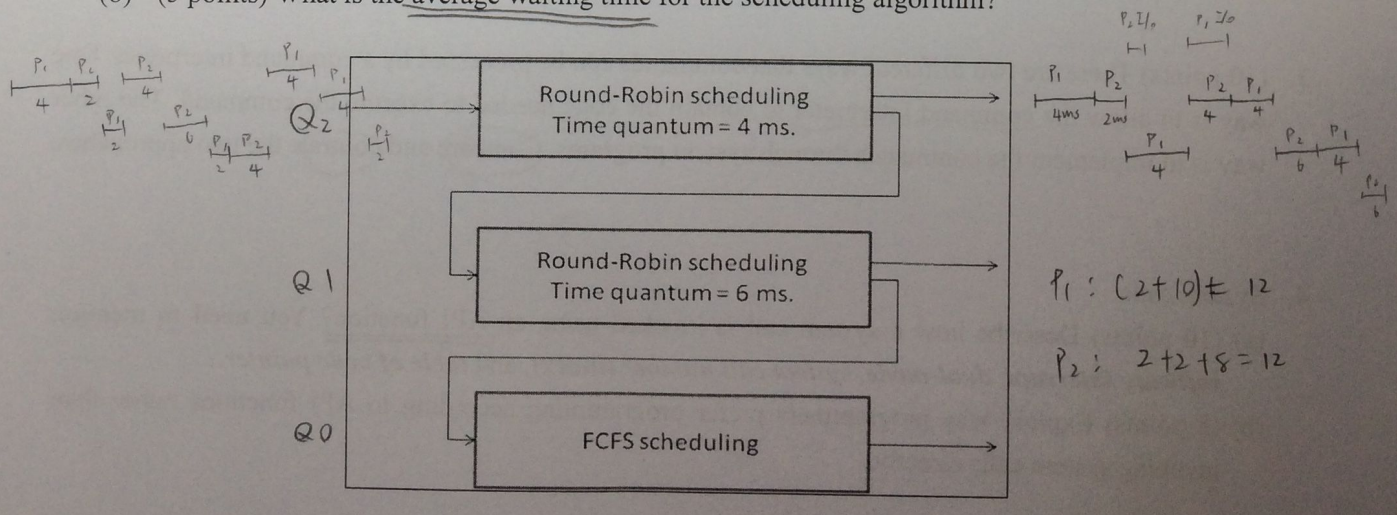
7. (5 points) Design a solution to avoid the starvation problem of priority scheduling algorithms

age

8. Scheduling – Consider the following two processes P_1 and P_2, each of which consists of two CPU bursts and one I/O burst. Assume that P_1 and P_2 arrive at time **0** and **2** (millisecond) respectively:

P_1:	CPU burst: 8 milliseconds.	I/O burst: 4 milliseconds.	CPU burst: 8 milliseconds.
P_2:	CPU burst: 2 milliseconds.	I/O burst: 2 milliseconds.	CPU burst: 16 milliseconds.

   (a) (10 points) Draw a Gantt chart illustrating the execution of the processes using the following **preemptive multilevel feedback-queue scheduling algorithm. Note that a high-level process will preempt a low-level process and a process will be scheduled to a lower level if it is preempted.**
   (b) (5 points) What is the average waiting time for the scheduling algorithm?



P1 : (2+10)+ 12

P2 : 2+2+8 = 12

9. (15 points) Please complete the following program that calculates equation 2x+3y-5. The values of variables x and y are provided by users in the command line, i.e., x = atoi(argv[1]) and y = atoi(argv[2]):

Hint: the main program first allocates a shared memory and then creates two child processes - one for the x term calculation and one for the y term calculation. Save the results calculated in the child processes in the shared memory and print the final outcome in the parent process.

```
int main(int argc, char *argv[])
{
    int x = atoi(argv[1]);   // the value of variable x
    int y = atoi(argv[2]);   // the value of variable y
    int i;
    pid_t pid;
    const int sh_size = 4;   // shared memory size = 4 bytes

    // allocate a shared memory segment
    int segment_id = shmget ( IPC_PRIVATE, sh_size, S_IRUSR | S_IWUSR )
    // attach the shared memory segment
    int *shared_memory = (int *) shmat( id , null , 0)

    *shared_memory = 0;  // initialization

    for(i = 2; i > 0; i--) // create two child processes
    {
    pid = fork( )

    if(pid < 0) {
       fprintf(stderr, "Fork Failed");
       exit(-1);

    } else if(pid == 0 ) {  // child part
       if(i==0)    // do x term calculation
         *shared_memory += 2 * x
       if(i==1)    // do y term calculation
         *shared_memory += 3 * y

       // detach the shared memory segment and exit
        shmdt ( shared_memory)
        exit(0)

    } else {  // parent part
       wait();
    }
    }

    printf("the result is %d\n",*shared_memory);

    // detach the shared memory segment
    shmdt(
    // remove the shared memory segment
    shmctl( shaid , IPC_RMID, NULL)

    return 0;

}
```